

Métodos de Otimização em Contexto Industrial

Otimização não linear diferenciável sem restrições

MATLAB - Comando `fminunc`

Ana Maria A. C. Rocha

Departamento de Produção e Sistemas

Universidade do Minho

arocha@dps.uminho.pt

Ano letivo 2023/24

Comando fminunc

- calcula o mínimo de problemas de otimização diferenciáveis
- mínimo de uma função multivariável sem restrições
- variáveis de números reais
- usa o método de 'quasi-Newton' ou 'trust-region'.

Para ver as opções disponíveis no algoritmo por defeito ('quasi-newton'):

```
optimoptions('fminunc')
```

Para ver a estrutura das opções de otimização disponíveis:

```
optimset('fminunc')
```

Para ver as opções de um algoritmo específico, p.ex. 'trust-region':

```
optimoptions('fminunc','Algorithm','trust-region')
```

Sintaxe de utilização

```
[x,fval,exitflag,output] = fminunc(fun,x0,opt)
```

Argumentos de entrada:

fun - é a função objetivo a minimizar

x0 - é o ponto inicial

opt - opções de controlo do processo de otimização.

- Especificar **fun** como um identificador de função para um ficheiro:

```
x = fminunc(@fun,x0)
```

em que o ficheiro **fun** é uma função MATLAB® definida por

```
function f = fun(x)
```

```
    f = ... ;
```

```
end
```

- Especificar **fun** como um identificador de função anónima, p.ex.:

```
x = fminunc(@(x)x(1)^2+sin(x(2)),x0);
```

Sintaxe de utilização: opt

Algorithm	Escolhe o algoritmo a usar: 'quasi-newton' (default) 'trust-region' (requer que seja dado o gradiente).
Display	Nível de apresentação off - não apresenta nada iter - resultados em cada iteração e uma mensagem de iter-detailed - resultados em cada iteração e uma me notify - resultado se a função não convergir e uma men notify-detailed - resultado se a função não convergir final - (default) apresenta apenas o resultado final e um final-detailed - apresenta apenas o resultado final e u
FunValCheck	Verifica se os valores da função objetivo são válidos on - apresenta erro se complex, Inf ou NaN. off - (default) não apresenta erro
MaxFunEvals	Nº máximo de cálculos da função (default=100*NumVar)
MaxIter	Nº máximo de iterações (default=400)

Sintaxe de utilização: opt

PlotFcn	Representa graficamente medidas do progresso do algoritmo. @optimplotx - representa o ponto atual. @optimplotfunccount - representa o número de cálculos @optimplotfval - representa o valor da função objetivo. @optimplotstepsize - representa o comprimento do passo @optimplotfirstorderopt - representa a medida de otimização
GradObj	Gradiente da função objetivo definido pelo utilizador. on - usa o gradiente definido pelo utilizador. off - (default) aproxima o gradiente por diferenças finitas
TolFun	Tolerância de paragem da função objetivo (default = 1e-6)
TolX	Tolerância de paragem relativamente a x (default=1e-6)
Hessian	Definição da Hessiana (default=off aproxima por dif. finitas)
HessUpdate	Método para escolher a direção de procura 'quasi-Newton'. bfgs - (default) aproxima a Hessiana pela fórmula BFGS lbfgs - aproxima a Hessiana pela Low-memory BFGS (large memory) dfp - aproxima a Hessiana pela fórmula DFP steepdesc - descida máxima

Sintaxe de utilização

```
[x,fval,exitflag,output] = fminunc(fun,x0,opt)
```

Argumentos de saída:

x - é a solução do problema.

fval - é o valor da função objetivo em **x**.

exitflag - descreve valores de saída do **fminunc**.

output - estrutura com informação acerca do processo de otimização

exitflag

- | | |
|----|---|
| 1 | a magnitude do vetor gradiente é menor que a tolerância de otimalidad |
| 2 | um alteração em x é menor que a tolerância TolX . |
| 3 | uma alteração na função objetivo é menor que a tolerância TolFun . |
| 5 | um previsto decréscimo na função objetivo é menor que a tolerância Tol |
| 0 | o número de iterações excedeu o MaxIter ou o MaxFunEvals . |
| -1 | indica que a função não convergiu para uma solução. |
| -3 | o problema parece ser não limitado. |

```
[x,fval,exitflag,output] = fminunc(fun,x0,opt)
```

output

iterations	indica o número de iterações realizadas
funcCount	indica o número de cálculos da função
firstorderopt	indica o valor da medida de otimalidade de 1ª ordem (norma do gradiente na solução em x)
algorithm	indica o algoritmo usado
cgiterations	nº total de iterações PCG (apenas para 'trust-region')
lssteplength	comprimento do passo da procura direta relativamente à direção de procura (apenas para 'quasi-newton')
stepsize	indica o comprimento do passo final em x
message	indica a mensagem de saída.

Algoritmos de otimização

Quasi-Newton Algorithm

O algoritmo quasi-Newton usa a fórmula de atualização BFGS (por defeito) para aproximar a matriz Hessiana e um método de procura linear cúbica.

Trust Region Algorithm

O algoritmo 'trust-region' requer que seja fornecido o vetor gradiente da função objetivo. É baseado no método de Newton 'interior-reflective', cuja cada iteração envolve a solução aproximada de um sistema linear de grandes dimensões usando o método dos gradientes conjugados pré-condicionados (PCG).

Síntese

- Sem mudar qualquer opção $\Leftrightarrow$ **Método quasi-Newton** (matriz Hessiana aproximada pela fórmula BFGS)
- Usando $\text{GradObj} = \text{'on'}$ $\Leftrightarrow$ **Método 'trust-region'**

Exercício 1

Calcule o mínimo do seguinte problema de otimização sem restrições, usando o algoritmo quasi-Newton

$$f(x_1, x_2) = x_1^2 + x_2^2 - x_1x_2.$$

Inicie o processo iterativo com o ponto $(1, 0)$.

- ❶ Sem usar qualquer opção.
- ❷ Visualizando os resultados obtidos em cada iteração.
- ❸ Representando graficamente os valores da função objetivo ao longo das iterações.
- ❹ Usando como tolerância de paragem $TolX = 10^{-10}$ e $TolFun = 10^{-12}$.
- ❺ Usando a fórmula de atualização DFP.
- ❻ Usando como tolerância de paragem 20 iterações.
- ❼ Usando como tolerâncias de paragem $TolX = 10^{-4}$ e 50 como máximo de iterações.
- ❽ Usando $TolFun = 10^{-5}$, 50 iterações no máximo e a fórmula DFP.

Exercício 2

O lucro da colocação de um sistema elétrico é dado por

$$\mathcal{L}(x_1, x_2) = 20x_1 + 26x_2 + 4x_1x_2 - 4x_1^2 - 3x_2^2$$

em que x_1 e x_2 são o custo da mão de obra e do material, respetivamente. Calcule o lucro máximo usando o método quasi-Newton, iniciando o processo com $x^{(1)} = (0, 0)$:

- ❶ usando o método quasi-Newton e a fórmula de atualização BFGS
- ❷ usando a fórmula de atualização DFP (da matriz Hessiana)
- ❸ usando a opção da representação gráfica dos valores da função objetivo, ao longo das iterações.
- ❹ usando $tolX = 10^{-2}$ e a fórmula de atualização BFGS
- ❺ usando $tolX = 10^{-2}$ e a fórmula de atualização DFP
- ❻ usando $TolFun = 10^{-12}$ e a fórmula BFGS e visualizando os resultados por iteração.

Exercício 3

A soma de três números (x_1, x_2 e x_3) positivos é igual a 40. Determine esses números de modo que a soma dos seus quadrados seja mínima. Use a relação da soma para colocar x_3 em função das outras 2 variáveis. Resolva o problema, iniciando o processo iterativo com a aproximação inicial $(x_1, x_2)^{(1)} = (10, 10)$:

- ① usando o método quasi-Newton e a fórmula de atualização da Hessiana BFGS
- ② usando o método quasi-Newton e a fórmula de atualização DFP
- ③ usando o método quasi-Newton, fórmula BFGS e $tolX = 10^{-4}$
- ④ usando o método quasi-Newton, $tolfun = 10^{-8}$, $tolx = 10^{-12}$ e a fórmula de atualização DFP.

Exercício 4

Suponha que pretendia representar um número A positivo como uma soma de três números positivos x_1, x_2 e x_3 . Para $A = 180$, determine esses números de tal forma que o seu produto seja o maior possível. Use a restrição para colocar x_3 em função das outras 2 variáveis. Resolva o problema a partir da aproximação inicial $(x_1, x_2)^{(1)} = (55, 55)$.

- ❶ usando o método quasi-Newton, fórmula BFGS
- ❷ usando o método quasi-Newton, fórmula de atualização DFP e $tolx = 10^{-4}$
- ❸ usando o método quasi-Newton, fórmula BFGS e $tolfun = 10^{-5}$
- ❹ usando o método quasi-Newton, $tolfun = 10^{-8}$, $tolx = 10^{-4}$ e representando graficamente os valores da função objetivo ao longo das iterações.

Resolução: Exercício 1

Representação gráfica da função: `ezsurf('x^2+y^2-x*y', [-100,100])`

- 1 Sem usar qualquer opção.

```
[x,fval,exitflag,output]=fminunc(@QN,[1,0])  
function [f] = fun(x)  
    f=@(x)x(1)^2+x(2)^2-x(1)*x(2);  
end
```

Nota: por defeito usa o método quasi-Newton e a fórmula de atualização BFGS.
ou

```
[x,fval,exitflag,output]=fminunc(@(x)x(1)^2+x(2)^2-x(1)*x(2), [1,0])
```

- 2 Visualizando os resultados obtidos em cada iteração.

```
op=optimset('Display','iter');  
[x,fval,exitflag,output]=fminunc(@(x)x(1)^2+x(2)^2-x(1)*x(2), [1,0], op)
```

- 3 Representando graficamente os valores da função objetivo ao longo das iterações.

```
op=optimset('PlotFcns',@optimplotfval);  
[x,fval,exitflag,output]=fminunc(@(x)x(1)^2+x(2)^2-x(1)*x(2), [1,0], op)
```

Resolução: Exercício 1 (cont.)

- 4 Usando como tolerância de paragem $TolX = 10^{-10}$ e $TolFun = 10^{-12}$.

```
op=optimset('TolX',1e-10,'TolFun',1e-12);  
[x,fval,exitflag,output]=fminunc(@(x)x(1)^2+x(2)^2-x(1)*x(2),[1,0],op)
```

- 5 usando a fórmula de atualização DFP.

```
op=optimset('hessupdate','dfp')  
[x,fval,exitflag,output]=fminunc(@(x)x(1)^2+x(2)^2-x(1)*x(2),[1,0],op)
```

- 6 Usando como tolerância de paragem 20 iterações.

```
op=optimset('MaxIter',20);  
[x,fval,exitflag,output]=fminunc(@(x)x(1)^2+x(2)^2-x(1)*x(2),[1,0],op)
```

- 7 Usando como tolerâncias de paragem $TolX = 10^{-4}$ e 50 como máximo de iterações.

```
op=optimset('TolX',1e-3,'MaxIter',50);  
[x,fval,exitflag,output]=fminunc(@(x)x(1)^2+x(2)^2-x(1)*x(2),[1,0],op)
```

- 8 Usando $TolFun = 10^{-5}$, 50 como máximo de iterações e a fórmula DFP.

```
op=optimset('TolFun',1e-5,'MaxIter',50,'HessUpdate','DFP');  
[x,fval,exitflag,output]=fminunc(@(x)x(1)^2+x(2)^2-x(1)*x(2),[1,0],op)
```

Resolução: Exercício 2

$$\begin{aligned}\max \mathcal{L}(x_1, x_2) &= -\min(-\mathcal{L}(x_1, x_2)) \\ \min &-(20x_1 + 26x_2 + 4x_1x_2 - 4x_1^2 - 3x_2^2)\end{aligned}$$

Representação gráfica da função: `ezsurf('20*x+26*y+4*x*y-4*x^2-3*y^2')`
`ezcontourf('20*x+26*y+4*x*y-4*x^2-3*y^2', [-20,20])`

- 1 usando o método quasi-Newton e a fórmula de atualização BFGS

```
[x,fval,exitflag,output]=fminunc(@QN1,[0,0])  
function [f] = QN1(x)  
    f=-(20*x(1)+26*x(2)+4*x(1)*x(2)-4*x(1)^2-3*x(2)^2);  
end
```

- 2 usando a fórmula de atualização DFP (da matriz Hessiana)

```
op=optimset('hessupdate','dfp')  
[x,fval,exitflag,output]=fminunc(@QN1,[0,0],op)  
function [f] = QN1(x)  
    f=-(20*x(1)+26*x(2)+4*x(1)*x(2)-4*x(1)^2-3*x(2)^2);  
end
```

- 3 usando a opção da representação gráfica dos valores da função objetivo, ao longo das iterações (`optimset('PlotFcns',@optimplotfval)`).

```
op=optimset('PlotFcn',@optimplotfval);  
[x,fval,exitflag,output]=fminunc(@QN1,[0,0],op)
```

Resolução: Exercício 2 (cont.)

- ① usando $tolX = 10^{-2}$ e a fórmula de atualização BFGS

```
op=optimset('tolx',1e-2);  
[x,fval,exitflag,output]=fminunc(@QN1,[0,0],op)  
function [f] = QN1(x)  
    f=-(20*x(1)+26*x(2)+4*x(1)*x(2)-4*x(1)^2-3*x(2)^2);  
end
```

- ② usando $tolX = 10^{-2}$ e a fórmula de atualização DFP

```
op=optimset('tolx',1e-2,'hessupdate','dfp');  
[x,fval,exitflag,output]=fminunc(@QN1,[0,0],op)  
function [f] = QN1(x)  
    f=-(20*x(1)+26*x(2)+4*x(1)*x(2)-4*x(1)^2-3*x(2)^2);  
end
```

- ③ usando $TolFun = 10^{-12}$, a fórmula BFGS e visualizando os resultados por iteração

```
op=optimset('TolFun',1e-12,'Display','iter');  
[x,fval,exitflag,output]=fminunc(@QN1,[0,0],op)  
function [f] = QN1(x)  
    f=-(20*x(1)+26*x(2)+4*x(1)*x(2)-4*x(1)^2-3*x(2)^2);  
end
```


Resolução: Exercício 3

Sabemos que $x_1 + x_2 + x_3 = 40$ e queremos minimizar $x_1^2 + x_2^2 + x_3^2$, logo fazendo $x_3 = 40 - x_1 - x_2$

$$\min_{x \in \mathbb{R}^2} f(x_1, x_2) \equiv x_1^2 + x_2^2 + (40 - x_1 - x_2)^2$$

Representação gráfica da função: `ezsurf('x^2+y^2+(40-x-y)^2', [-100,100])`

- ❶ usando o método quasi-Newton e a fórmula de atualização da Hessiana BFGS

```
[x,fval,exitflag,output]=fminunc(@(x)x(1)^2+(2)^2+(40-x(1)-x(2))^2,[10
```

- ❷ usando o método quasi-Newton e a fórmula de atualização DFP

```
op=optimset('hessupdate','dfp')
```

```
[x,fval,exitflag,output]=fminunc(@(x)x(1)^2+(2)^2+(40-x(1)-x(2))^2,[10
```

- ❸ usando o método quasi-Newton, fórmula BFGS e $tolX = 10^{-4}$

```
op=optimset('tolx',1e-4);
```

```
[x,fval,exitflag,output]=fminunc(@(x)x(1)^2+(2)^2+(40-x(1)-x(2))^2,[10
```

- ❹ usando o método quasi-Newton, $tolfun = 10^{-8}$, $tolx = 10^{-12}$ e a fórmula de atualização DFP

```
op=optimset('tolfun',1e-8,'tolx',1e-12,'hessupdate','dfp');
```

```
[x,fval,exitflag,output]=fminunc(@(x)x(1)^2+(2)^2+(40-x(1)-x(2))^2,[10
```

Resolução: Exercício 4

Sabemos que $A = 180 = x_1 + x_2 + x_3$ e queremos maximizar $x_1 x_2 x_3$, logo fazendo $x_3 = 180 - x_1 - x_2$

$$\max_{x \in \mathbb{R}^2} f(x_1 x_2) \equiv x_1 x_2 (180 - x_1 - x_2) \Leftrightarrow \min_{x \in \mathbb{R}^2} - (x_1 x_2 (180 - x_1 - x_2))$$

Representação gráfica da função: `ezsurf('x*y*(180-x-y)')`

- ❶ usando o método quasi-Newton e a fórmula de atualização da Hessiana BFGS

```
[x,fval,exitflag,output]=fminunc(@(x)-(x(1)*x(2)*(180-x(1)-x(2)))), [55,
```

- ❷ usando o método quasi-Newton, a fórmula de atualização DFP e $tolX = 10^{-4}$

```
op=optimset('hessupdate','dfp','tolx'1e-4)
```

```
[x,fval,exitflag,output]=fminunc(@(x)-(x(1)*x(2)*(180-x(1)-x(2)))), [55,
```

- ❸ usando o método quasi-Newton, fórmula BFGS e $tolfun = 10^{-5}$

```
op=optimset('tolfun',1e-5);
```

```
[x,fval,exitflag,output]=fminunc(@(x)-(x(1)*x(2)*(180-x(1)-x(2)))), [55,
```

- ❹ usando o método quasi-Newton, $tolfun = 10^{-8}$, $tolx = 10^{-12}$ e representando graficamente os valores da função objetivo ao longo das iterações

```
op=optimset('tolfun',1e-8,'tolx',1e-12,'PlotFcns',@optimplotfval);
```